Module : 04

Section-A Complexity Analysis of previous Module

Module 1 - Patterns Programs

1. SQUARE PATTERN :→

- Time Complexity : $O(N^2)$ Outer loop runs N times; inner loop also runs N times, Total operation : $(N \times N) = N^2$
- Space Complexity : $O(1)$ only loop counters i, j and n are stored - no extra data structure grows with N.
- Optimized approach : Not really optimizable further - you must physically output N^2 characters, so $O(N^2)$ is inherent to output size itself.

2. Hollow Square Pattern :→

- Time Complexity : $O(N^2)$ - Same nested loop structure as the square the 'if' condition checking border cells is $O(1)$ work per cell, so still is N^2
- Space complexity : $O(1)$ - Fixed variables only

- Optimized approach :- None - needed

3. Right Triangle (* Increasing per row)

- Time Complexity :- $O(N^2)$ Outer loop runs N times, inner loop on row i runs i+1 times. Total stars printed = $1+2+3+\cdots+N$ $= N(N+1)/2 = O(N^2)$

- Space complexity :- $O(1)$

- Optimized approach :- Already minimal - no further approach exists.

4. INVERTED TRIANGLE :-

- Time Complexity :- $O(N^2)$ - Same reasoning as right triangle but counting down; sum of decreasing row-lengths is still $O(N^2)$.

- Space complexity :- $O(1)$

5. Pyramid Pattern :-

- Time complexity :- $O(N^2)$ Two inner loops per row (spaces + stars), but both are bounded by N. So total work $O(N^2)$

- Space complexity :- $O(1)$

6. DIAMOND PATTERN

- Time complexity : $O(N^2)$ - upper pyramid ($O(N^2)$) + lower inverted pyramid ($O(N^2)$) = $O(N^2)$ overall.
- Space Complexity: $O(1)$

Note :- Same complexities for Half Diamond and Half Inverted Diamond.

NUMBER PATTERNS :-

- Time Complexity : $O(N^2)$ for all - every one uses a double nested loop where both loops are bounded by N. so total printed elements scale with N^2.
- Space complexity :- $O(1)$ for all - only counters and a constant number of extra int variables are used.
- Optimized approach :- None of these can go below $O(N^2)$ since the very definition of the pattern requires printing ~ $N^2/2$ to N^2.

Module :- 2 — SEARCHING & SORTING BASICS

1. LINEAR SEARCH

for(i=0 to size-1) if (arr[i]==target) return;

- Time Complexity :- O(N) in the worst case the loop checks every one of the N elements exactly once before concluding.
- Space complexity :- O(1) NO extra structures are created.
- Optimized approach :- If the array is sorted, Binary Search (O(logN)) is far more efficient - otherwise linear search is optimal for unsorted data

2. BINARY SEARCH

while(st<=end) {mid = ...; compare; shrink range}

- Time Complexity :- O(logN) each iteration eliminates half of the remaining search page, so the number of iterations needed to shrink N elements to 1 is $\log_2 N$.
- Space complexity :- O(1) Only st, end & mid are maintained - no recursion stack or extra array.

- Optimized approach : Already optimal for comparison-based sorting a sorted array - O(logN) is the theoretical lower bound for this problem.

3. BUBBLE SORT

```
for (i=0... n-2) for (j=0... n-i-2)
    if (arr[j] > arr[j+1]) swap
```

- Time Complexity : O(N²) worst/average case; O(N) best case (already sorted, due to the isswap early-exist flag) The outer loop runs N-1 times and, for each pass, the inner loop compares adjacent elements up to N-i-1 times giving roughly N²/2 comparisons in worst case. The isswap flag lets it terminate early on a sorted/near-sorted array, giving the O(N) best case.
- Space Complexity: O(1) Sorting is done in place using a temporary variable during swaps.
- optimized approach: Merge sort or Quick sort (O(NlogN)) are far more efficient for large, unsorted datasets - bubble sort's repeated adjacent-swapping is inherently quadratic

4. SELECTION SORT

for (i=0.. n-2) find min in rem, swap into place.

- TC ÷ $O(N^2)$ in all cases (best, average, worst) for every element at position i, the inner loop scans the remaining N-i elements to find the minimum - this happens regardless of the input's initial order.
- SC ÷ O(1) - sorting is in place with just one swap per outer iteration.
- Optimized approach: Merge Sort / Heap Sort ($O(N \log N)$) are preferable since selection sort does a full O(N) scan on every outer iteration regardless of how sorted the array already is.

5. INSERTION SORT

for (i=1 .... n-1) shift longer elements right, insert curr in place.

- Time Complexity ⇒ $O(N^2)$ worst / average case; O(N) best case (already sorted) Each element is compared against the sorted prefix to its left.

in the worst case (reverse sorted), this requires shifting up to i elements for each of the N elements. If the array is already sorted, the while condition fails immediately each time, giving linear time.

- Space Complexity: $O(1)$
- Optimized approach: MergeSort ($O(N \log N)$) is better; however insertion sort remains efficient for small or nearly-sorted arrays.

Module:3 - MERGE SORT, QUICKSORT, HEAPSORT.

1. MERGE SORT

mergesort(arr, start, end): split at mid, recursive both halves, merge

- TC :→ O(N log N) in all (best/avg/worst) case. The array is recursively split in half, giving a recursion depth of log N: at each of (log N levels, the merge step does O(N) work combining sorted halves. Total = O(N) x O(log N) = O(N log N)

- SC :→ O(N) The merge function allocates a temporary vector (temp) to hold merged results at every call, and although these are released after each merge, the recursion stack plus temporary storage in together require O(N) auxiliary space.

- Optimized approach :→ Heap-sort.

2. QUICK SORT

quicksort (arr, st, end) : pick pivot, partition, recurse on both sides.

- TC → $O(N \log N)$ average case; $O(N^2)$ worst case On average, each partition step splits the array roughly in half, giving $\log N$ levels of recursion with $O(N)$ partitioning work per level = $O(N \log N)$. In the worst case each partition only removes one element, causing N levels of recursion = $O(N^2)$.

- SC → $O(\log N)$ average case (recursion stack depth):
$O(N)$ worst case NO extra array is used, but the recursive call stack grows with the recursion depth, which is $\log N$ on average and N in worst case

- Optimized approach : using randomized pivot selection avoids the worst-case $O(N^2)$ scenario on already-sorted or reverse-sorted inputs, keeping the expected complexity at $O(N \log N)$.

3. HEAP SORT :→

heapSort : build max-heap O(N), then repeatedly extract max & heapify

- Time Complexity :→ O(NlogN) in all cases. Building the initial max-heap takes O(N) time ; then each of the N extraction steps swaps the root to the end and calls heapify, which takes O(logN) time. Total = O(N) + O(NlogN) = O(NlogN)

- Space Complexity :→ O(1) Heap sort operates entirely in-place on the input array - no auxiliary array is needed, unlike merge sort.

- Optimized approach :→ It achieves optimal TC & SC among three.

Algorithm	Best	Avg	Worst	Space
Patterns	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Linear Search	$O(1)$	$O(N)$	$O(N)$	$O(1)$
Binary Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$ - $O(N)$
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$

Problem 1:- Print 1 to N

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin>>n;

    for (int i=1; i<=n; i++){
        cout<< i;
        if (i!=n) cout<< " ";
    }
    cout<< endl;
    return 0;

}
```

- Time Complexity :- O(N) number of operations grows linearly with the input size N, there's no nested iteration or repeated pass over the data.
- Space Complexity :- O(1)
- Optimized approach :- Already optimal.

Problem 2 : Sum of First N Natural no.s.

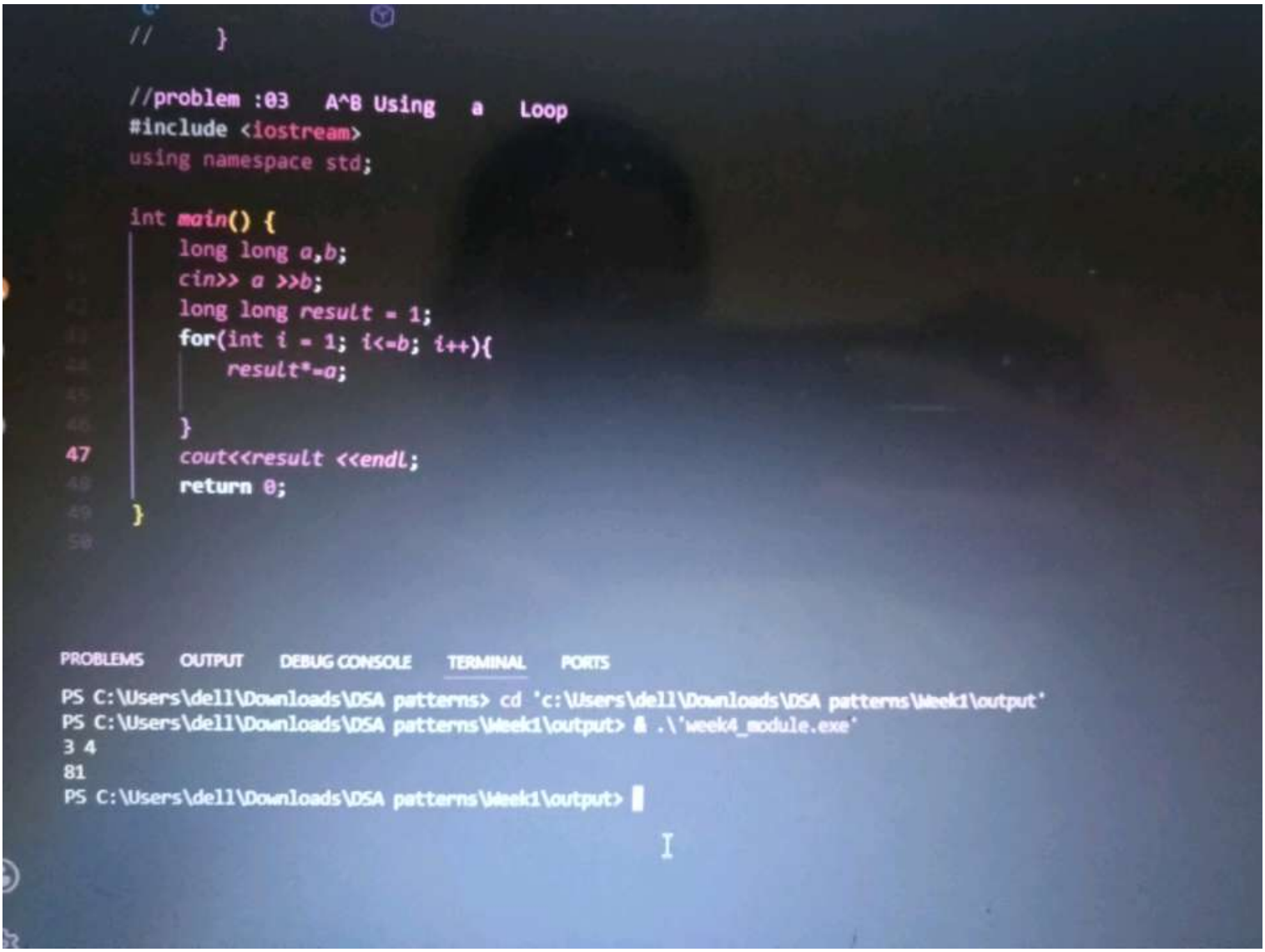
```
#include <iostream>
using namespace std;

int main(){
    int n;
    cin>>n;

    long long sum=0;
    for(int i=1;i<=n;i++){
        sum+=i;
    }
    cout<<sum<<endl;

    return 0;
}
```

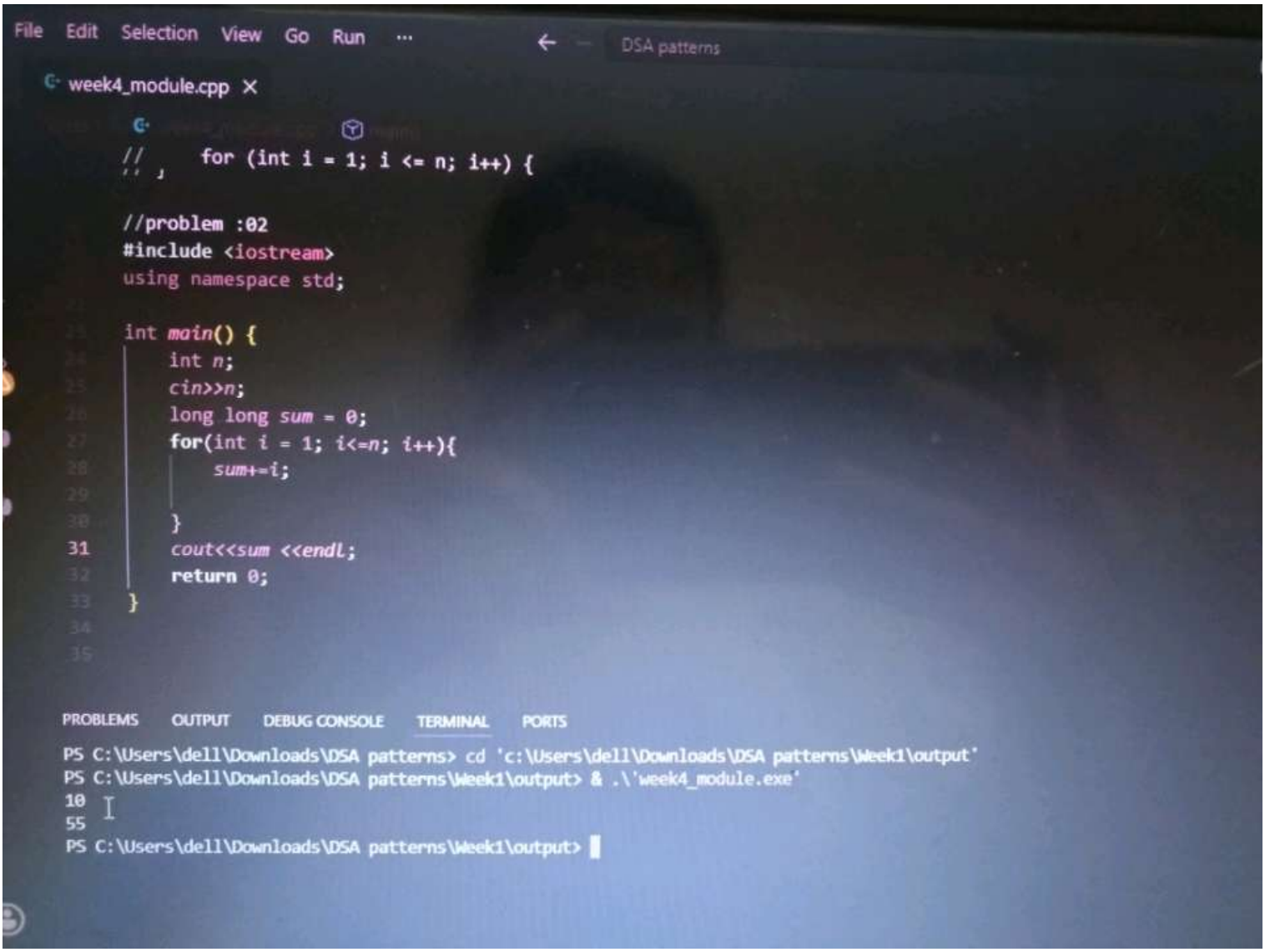
- TC :→ O(N)
- SC :→ O(1)
- Optimized approach : Yes

Problem 3 :→ A^B Using a loop :←

```
#include <iostream>
using namespace std;

int main(){
    int a,b;
    cin>>a>>b;

    long long result = 1;
    for(int i=0; i<b; i++){
        result *=a;
    }
    cout << result << endl;

    return 0;
}
```

- TC : O(B) The loop runs exactly B times (the exponent), multiplying the result by A on each pass. Work scales linearly with B not with A, so complexity is O(B).
- SC : O(1) Only a, b, result, and i are used, all fixed-size variables independent of the magnitude of A or B
- Optimized tech : Yes.


//   }
//problem :03   A^B Using   a   Loop
#include <iostream>
using namespace std;
int main() {
long long a,b;
cin>> a >>b;
long long result = 1;
for(int i = 1; i<=b; i++){
result*=a;
}
cout<<result <<endl;
return 0;
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\dell\Downloads\DSA patterns> cd 'c:\Users\dell\Downloads\DSA patterns\Week1\output'
PS C:\Users\dell\Downloads\DSA patterns\Week1\output> & .\'week4_module.exe'
3 4
81
PS C:\Users\dell\Downloads\DSA patterns\Week1\output>


File Edit Selection View Go Run ...
DSA patterns
week4_module.cpp
// for (int i = 1; i <= n; i++) {
//problem :02
#include <iostream>
using namespace std;
int main() {
int n;
cin>>n;
long long sum = 0;
for(int i = 1; i<=n; i++){
sum+=i;
}
cout<<sum <<endl;
return 0;
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\dell\Downloads\DSA patterns> cd 'c:\Users\dell\Downloads\DSA patterns\Week1\output'
PS C:\Users\dell\Downloads\DSA patterns\Week1\output> & .\'week4_module.exe'
10
55
PS C:\Users\dell\Downloads\DSA patterns\Week1\output>

week4_module.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {
        cout << i;
        if (i != n) {
            cout << " ";
        }
    }
    cout << endl;
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\dell\Downloads\DSA patterns> cd 'c:\Users\dell\Downloads\DSA patterns\Week1\output'
PS C:\Users\dell\Downloads\DSA patterns\Week1\output> & .\'week4_module.exe'
8
1 2 3 4 5 6 7 8
PS C:\Users\dell\Downloads\DSA patterns\Week1\output>
```